

# Artificial Neural Networks

**Sachchida Nand Chaurasia**

Assistant Professor

Department of Computer Science

Banaras Hindu University

Varanasi

Email id: [snchaurasia@bhu.ac.in](mailto:snchaurasia@bhu.ac.in), [sachchidanand.mca07@gmail.com](mailto:sachchidanand.mca07@gmail.com)



March 3, 2025

## Artificial Neuron I

- ✓ An artificial neuron is a connection point in an artificial neural network. Artificial neural networks, like the human body's biological neural network, have a layered architecture and each network node (connection point) has the capability to process input and forward output to other nodes in the network.
- ✓ In both artificial and biological architectures, the nodes are called neurons and the connections are characterized by synaptic weights, which represent the significance of the connection.
- ✓ As new data is received and processed, the synaptic weights change and this is how learning occurs.

## Artificial Neuron II

- ✓ In both artificial and biological networks, when neurons process the input they receive, they decide whether the output should be passed on to the next layer as input.
- ✓ The decision of whether or not to send information on is called bias and it's determined by an activation function built into the system.
- ✓ For example, an artificial neuron may only pass an output signal on to the next layer if its inputs (which are actually voltages) sum to a value above some particular threshold value.
- ✓ Because activation functions can either be linear or non-linear, neurons will often have a wide range of convergence and divergence.

## Artificial Neuron III

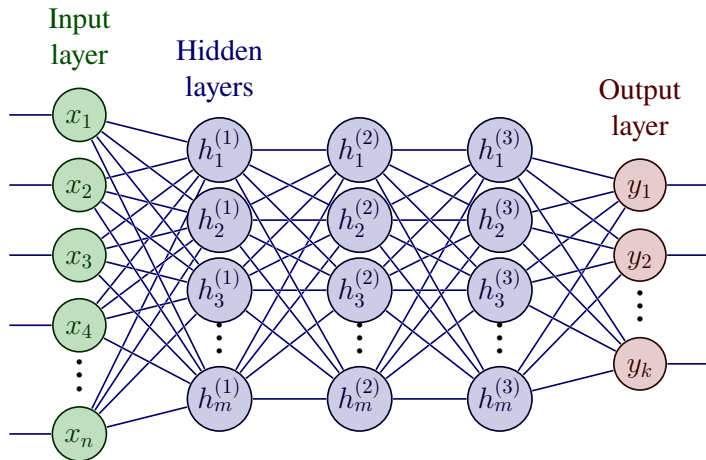
- ✓ Divergence is the ability for one neuron to communicate with many other neurons in the network and convergence is the ability for one neuron to receive input from many other neurons in the network.
- ✓ An **artificial neuron** is a mathematical function conceived as a model of biological neurons, a neural network.
- ✓ Artificial neurons are elementary units in an artificial neural network. The artificial neuron receives one or more inputs and sums them to produce an output (or activation, representing a neuron's action potential which is transmitted along its axon).

## Artificial Neuron IV

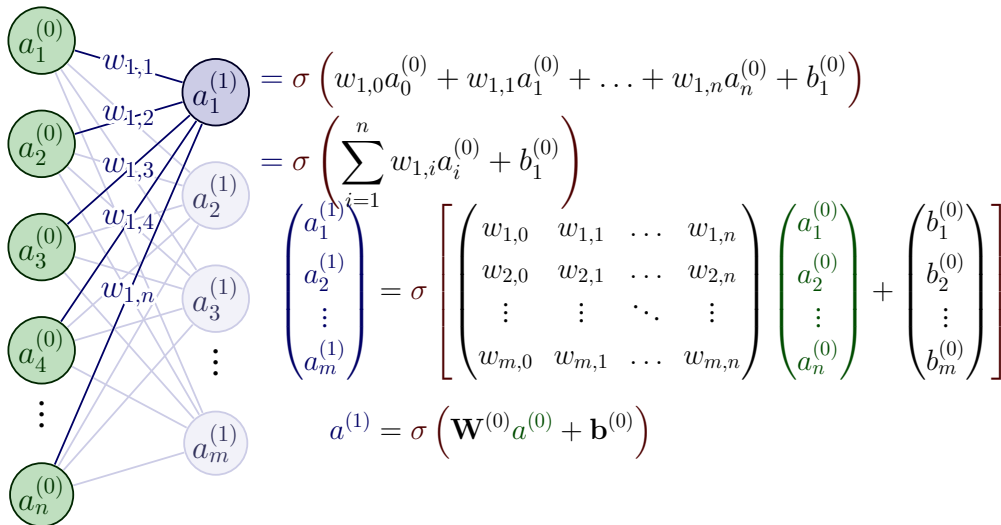
- ✓ Usually each input is separately weighted, and the sum is passed through a non-linear function known as an activation function or transfer function.
- ✓ The transfer functions usually have a sigmoid shape, but they may also take the form of other non-linear functions, piecewise linear functions, or step functions.
- ✓ They are also often monotonically increasing, continuous, differentiable and bounded.
- ✓ The thresholding function has inspired building logic gates referred to as threshold logic; applicable to building logic circuits resembling brain processing.

## Artificial Neuron V

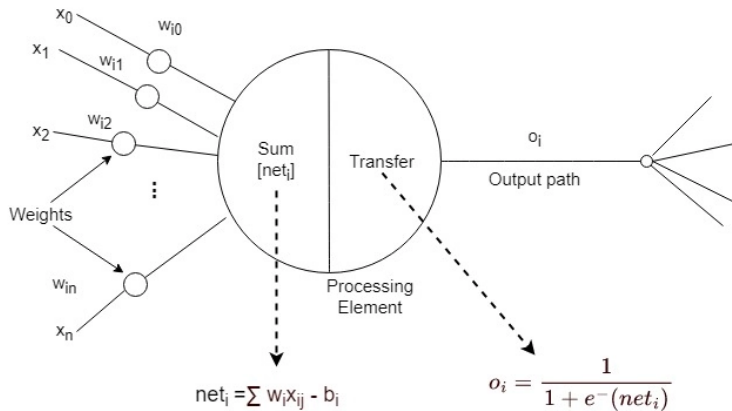
### Basic structure of artificial neural network



## Artificial Neuron VI



## Artificial Neuron VII



**Figure 1:** Basic Artificial Neuron

## Artificial Neuron VIII

For a given artificial neuron  $k$ , let there be  $m + 1$  inputs with signals  $x_0$  through  $x_m$  and weights  $w_{k0}$  through  $w_{km}$ .

✓ Usually, the  $x_0$  input is assigned the value +1, which makes it a bias input with  $w_{k0} = b_k$ . This leaves only  $m$  actual inputs to the neuron: from  $x_1$  to  $x_m$ .

✓ The output of the  $i^{th}$  neuron is:

$$y_k = \varphi \left( \sum_{j=0}^m w_{kj} x_j \right) \quad (1)$$

## Artificial Neuron IX

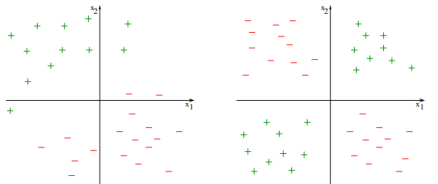
where  $\varphi$  (phi) is the transfer function (commonly a threshold function).

- ✓ The output is analogous to the axon of a biological neuron, and its value propagates to the input of the next layer, through a synapse.
- ✓ It may also exit the system, possibly as part of an output vector.
- ✓ It has no learning process as such. Its transfer function weights are calculated and threshold value are predetermined.

## Single and multilayer perceptrons I

### Perceptron convergence theorem:

The 'Perceptron convergence theorem' states that for any data set which is linearly separable the Perceptron learning rule is guaranteed to find a solution in a finite number of steps.



## Single and multilayer perceptrons II

**Figure 2:** Labeled points in 2-dimensional space. The data set on the left is linearly separable, whereas the data set on the right is not

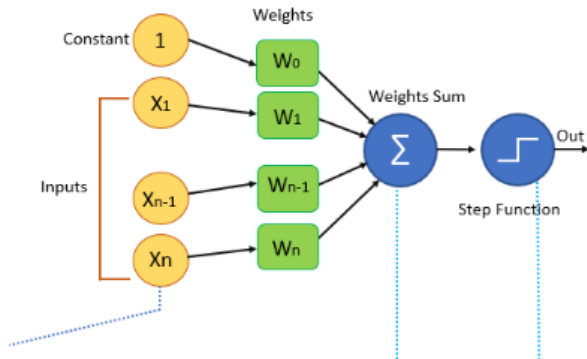
### Perceptron Learning Algorithm:

- ✓ Perceptron Algorithm is used in a supervised machine learning domain for classification.
- ✓ In classification, there are two types of linear classification and no-linear classification.
- ✓ Linear classification is nothing but if we can classify the data set by drawing a simple straight line then it can be called a linear binary classifier.

## Single and multilayer perceptrons III

✓ Whereas if we cannot classify the data set by drawing a simple straight line then it can be called a non-linear binary classifier.

## Single and multilayer perceptrons IV



**Figure 3:** Perceptron Algorithm Block Diagram

## Single and multilayer perceptrons V

- Perceptron Algorithm Block Diagram

- ➊ Input: All the features of the model we want to train the neural network will be passed as the input to it, Like the set of features  $[X_1, X_2, X_3 \dots X_n]$ . Where  $n$  represents the total number of features and  $X$  represents the value of the feature.
- ➋ Weights: Initially, we have to pass some random values as values to the weights and these values get automatically updated after each training error that is the values are generated during the training of the model. In some cases, weights can also be called as weight coefficients.
- ➌ Weights Sum: Each input value will be first multiplied with the weight assigned to it and the sum of all the multiplied values is known as a **weighted sum**.

## Single and multilayer perceptrons VI

- **Step or Activation Function:**

- ✓ Activation function applies step rule which converts the numerical value to 0 or 1 so that it will be easy for data set to classify.
- ✓ Based on the type of value we need as output we can change the activation function. Sigmoid function, if we want values to be between 0 and 1 we can use a sigmoid function that has a smooth gradient as well.
- ✓ Sign function, if we want values to be +1 and -1 then we can use sign function.

## Single and multilayer perceptrons VII

- ✓ The hyperbolic tangent function is a zero centered function making it easy for the multilayer neural networks.
- ✓ Relu function is highly computational but it cannot process input values that approach zero. It is good for the values that are both greater than and less than a Zero.

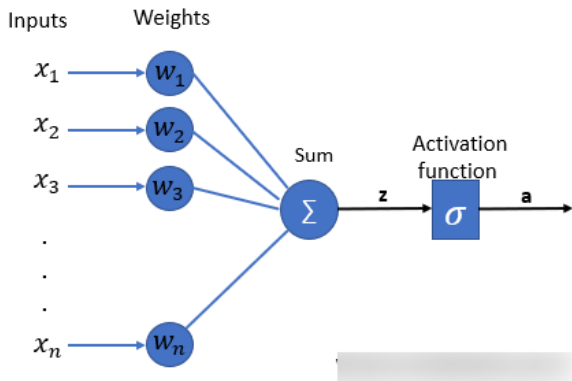
### **Bias:**

- ✓ The passed value one as input in the starting and  $W_0$  in the weights section  $W_0$  is an element that adjusts the boundary away from origin to move the activation function left, right, up or down.

## Single and multilayer perceptrons VIII

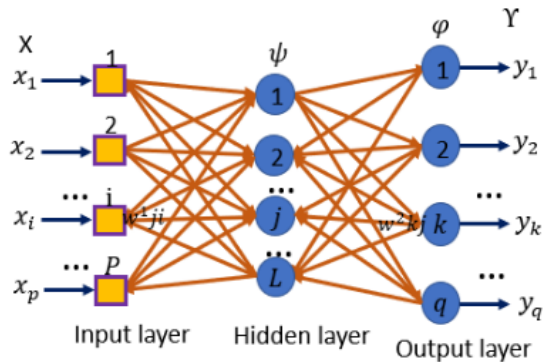
- ✓ Since we want this to be independent of the input features, we add constant one in the statement so the features will not get affected by this and this value is known as Bias.
- ✓ Perceptron algorithms can be divided into two types they are single layer perceptrons and multi-layer perceptron's.
- ✓ In single-layer perceptron's neurons are organized in one layer whereas in a multilayer perceptron's a group of neurons will be organized in multiple layers.
- ✓ Every single neuron present in the first layer will take the input signal and send a response to the neurons in the second layer and so on. 🤖

## Single and multilayer perceptrons IX



**Figure 4:** Single Layer Perceptron

## Single and multilayer perceptrons X



**Figure 5:** Multi-Layer Perceptron

## Single and multilayer perceptrons XI

- **Perceptron Learning Steps**

- ① Features of the model we want to train should be passed as input to the perceptrons in the first layer.
- ② These inputs will be multiplied by the weights or weight coefficients and the production values from all perceptrons will be added.
- ③ Adds the Bias value, to move the output function away from the origin.
- ④ This computed value will be fed to the activation function (chosen based on the requirement, if a simple perceptron system activation function is step function).
- ⑤ The result value from the activation function is the output value.

## Single and multilayer perceptrons XII

- ✓ The input to the Perceptron Learning Algorithm is a data set of  $n \geq 1$  points (each  $d$ -dimensional), and their associated labels (+ 'or' -).
- ✓ For mathematical convenience, we associate the '+' label with the number '+1', and the '-' label with the number '-1'.
- ✓ we may take our input to be:  $(x^1, y^1), (x^2, y^2), \dots, (x^n, y^n)$ , where for  $i \in \{1, 2, \dots, n\}$ ,  $x^i \in \mathbb{R}^d$  and  $y^i \in \{-1, 1\}$ .

**Assumption 1 (Linear Separability):** There exists some  $\mathbf{w}^* \in \mathbb{R}^d$  such that  $\|\mathbf{w}^*\| = 1$  and for some  $\gamma > 0$  for all  $i \in \{1, 2, \dots, n\}$ .

## Single and multilayer perceptrons XIII

**Assumption 2 (Bounded coordinates):** There exists  $R \in \mathbb{R}$  such that for  $i \in \{1, 2, \dots, n\}$ ,  $\|x^i\| \leq R$ .

## Number of hidden layers and nodes in a feedforward neural network? I

- ✓ Once the network is initialized, you can iteratively tune the configuration during training using a number of ancillary algorithms.
- ✓ one family of these works by pruning nodes based on (small) values of the weight vector after a certain number of training epochs—in other words, eliminating unnecessary/redundant nodes.
- ✓ Every Neural Network (NN) has three types of layers: input, hidden, and output.
- ✓ Creating the NN architecture therefore means coming up with values for the number of layers of each type and the number of nodes in each of these layers.

## Number of hidden layers and nodes in a feedforward neural network? II

### The Input Layer

- ✓ With respect to the number of neurons comprising this layer, this parameter is completely and uniquely determined once you know the shape of your training data. Specifically, the number of neurons comprising that layer is equal to the number of features (columns) in your data.
- ✓ Some NN configurations add one additional node for a bias term.

### The Output Layer

- ✓ Like the Input layer, every NN has exactly one output layer. Determining its size (number of neurons) is simple; it is completely determined by the chosen model configuration.

## Number of hidden layers and nodes in a feedforward neural network? III

- ✓ ✓ If the NN is a regressor, then the output layer has a single node.
- ✓ ✓ If the NN is a classifier, then it also has a single node unless softmax is used in which case the output layer has one node per class label in your model.

### The Hidden Layers:

- ✓ How many hidden layers? *If your data is linearly separable (which you often know by the time you begin coding a NN) then you don't need any hidden layers at all. Of course, you don't need an NN to resolve your data either, but it will still do the job.*

## Number of hidden layers and nodes in a feedforward neural network? IV

✓ Beyond that, there is a consensus is the performance difference from adding additional hidden layers: the situations in which performance improves with a second (or third, etc.) hidden layer are very few. *One hidden layer is sufficient for the large majority of problems.*

✓ So what about size of the hidden layer(s)—how many neurons? There are some empirically-derived rules-of-thumb, of these, the most commonly relied on is *'the optimal size of the hidden layer is usually between the size of the input and size of the output layers'*.

## Number of hidden layers and nodes in a feedforward neural network? V

✓ In sum, for most problems, one could probably get decent performance (even without a second optimization step) by setting the hidden layer configuration using just two rules:

- 1 Number of hidden layers equals one; and
- 2 The number of neurons in that layer is the mean of the neurons in the input and output layers.

## Number of hidden layers and nodes in a feedforward neural network? VI

### Optimization of the Network Configuration:

- ✓ Pruning describes a set of techniques to trim network size (by nodes not layers) to improve computational performance and sometimes resolution performance.
- ✓ The gist of these techniques is removing nodes from the network during training by identifying those nodes which, if removed from the network, would not noticeably affect network performance (i.e., resolution of the data).
- ✓ Even without using a formal pruning technique, you can get a rough idea of which nodes are not important by looking at your weight matrix after

## Number of hidden layers and nodes in a feedforward neural network? VII

training; look weights very close to zero—it's the nodes on either end of those weights that are often removed during pruning.

✓ Obviously, if you use a pruning algorithm during training then begin with a network configuration that is more likely to have excess (i.e., 'prunable') nodes.

✓ There's one additional rule of thumb that helps for supervised learning problems. You can usually prevent over-fitting if you keep your number of neurons below:

## Number of hidden layers and nodes in a feedforward neural network? VIII

$$N_h = \frac{N_s}{(\alpha * (N_i + N_o))} \quad (2)$$

$N_i$  = number of input neurons.

$N_o$  = number of output neurons.

$N_s$  = number of samples in training data set.

$\alpha$  = an arbitrary scaling factor usually 2-10.

## Back Propagation I

- ✓ After performing the first pass (based on the input and randomly given inputs) error will be calculated and the back propagation algorithm performs an iterative backward pass and try to find the optimal values for weights so that the error value will be minimized.
- ✓ To minimize the error, back propagation algorithm will calculate partial derivatives from the error function till each neuron's specific weight, this process will give us complete transparency from total error value to a specific weight that is responsible for the error.

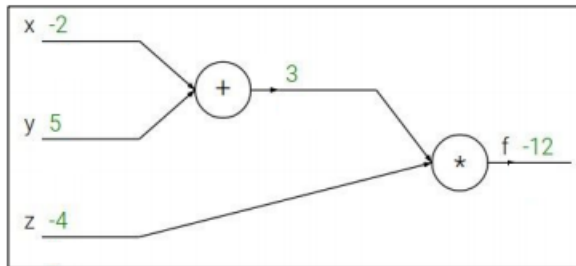
## Back Propagation II

### Example1:

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2$ ,  $y = 5$ ,  $z = -4$



## Back Propagation III

Backpropagation: a simple example

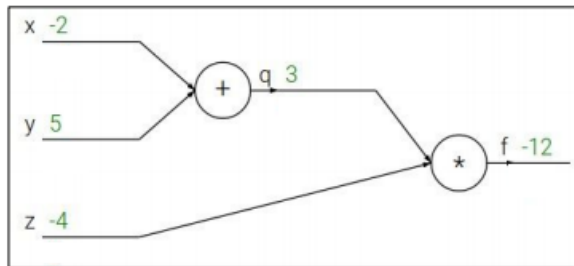
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2$ ,  $y = 5$ ,  $z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want:  $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



## Back Propagation IV

Backpropagation: a simple example

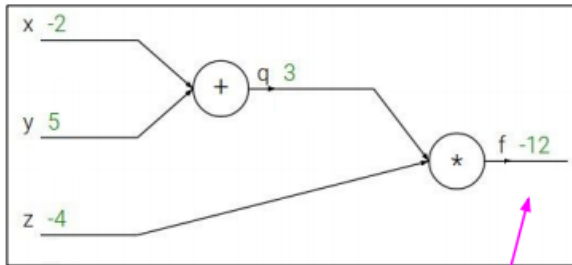
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2$ ,  $y = 5$ ,  $z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want:  $\frac{\partial f}{\partial x}$ ,  $\frac{\partial f}{\partial y}$ ,  $\frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial f}$$

## Back Propagation V

Backpropagation: a simple example

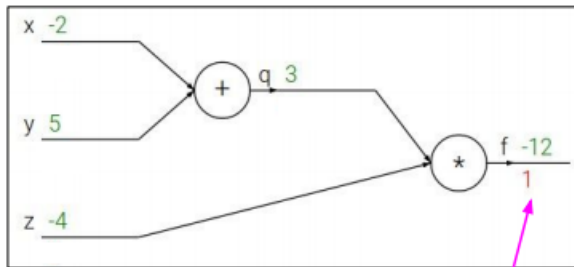
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want:  $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial f}$$

## Back Propagation VI

Backpropagation: a simple example

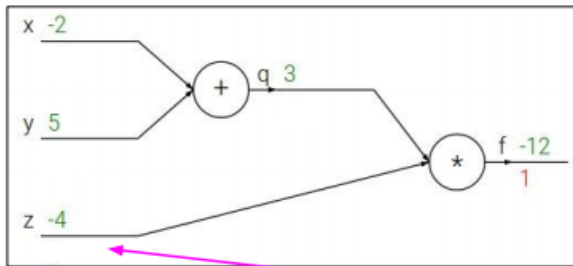
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2$ ,  $y = 5$ ,  $z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want:  $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

A pink arrow points from this box to the input  $z$  of the multiplication node in the computational graph above.

## Back Propagation VII

Backpropagation: a simple example

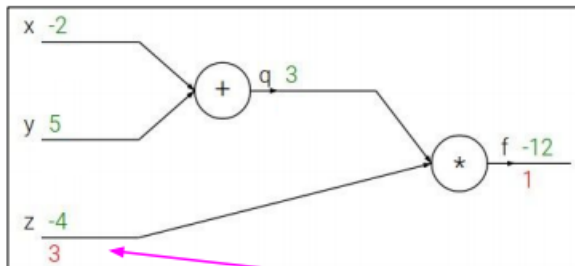
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want:  $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

## Back Propagation VIII

Backpropagation: a simple example

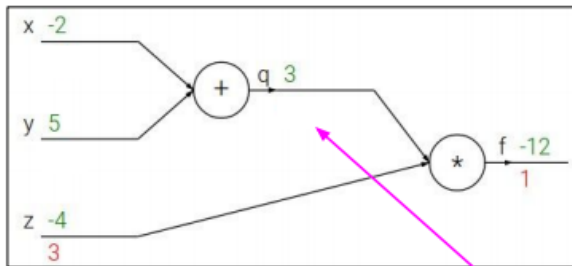
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2$ ,  $y = 5$ ,  $z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want:  $\frac{\partial f}{\partial x}$ ,  $\frac{\partial f}{\partial y}$ ,  $\frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial q}$$

## Back Propagation IX

Backpropagation: a simple example

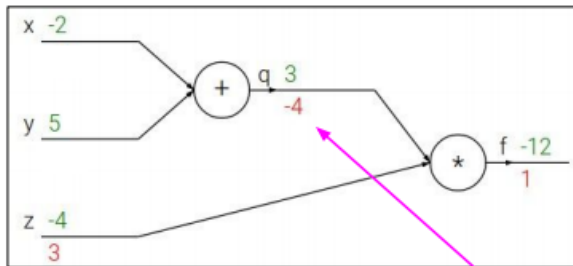
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want:  $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial q}$$

## Back Propagation X

Backpropagation: a simple example

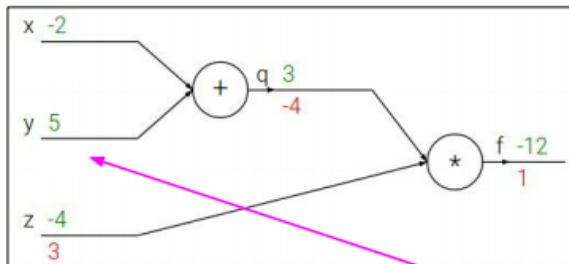
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want:  $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial y}$$

## Back Propagation XI

Backpropagation: a simple example

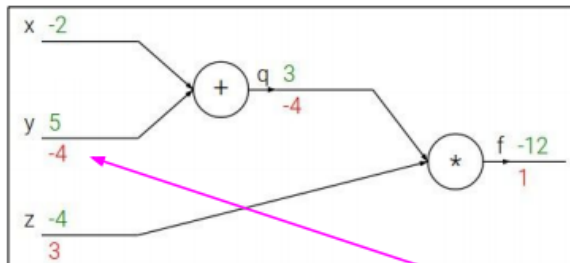
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want:  $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial y}$$

Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

## Back Propagation XII

Backpropagation: a simple example

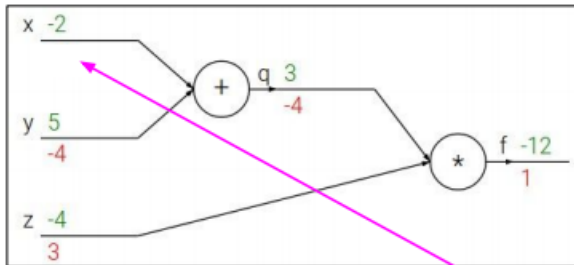
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want:  $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial x}$$

## Back Propagation XIII

Backpropagation: a simple example

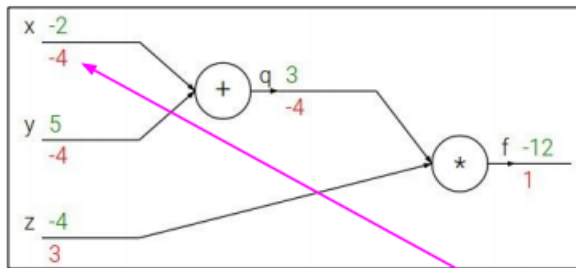
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want:  $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial x}$$

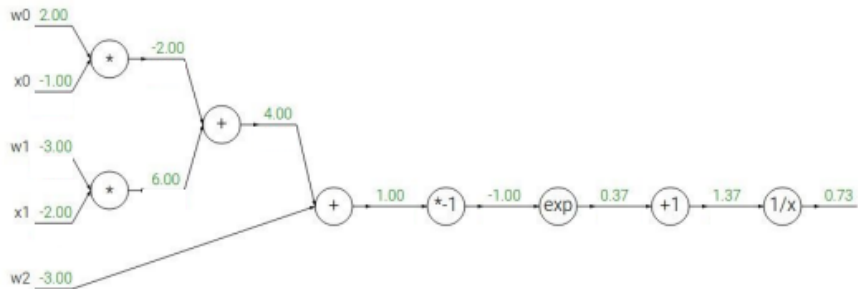
Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

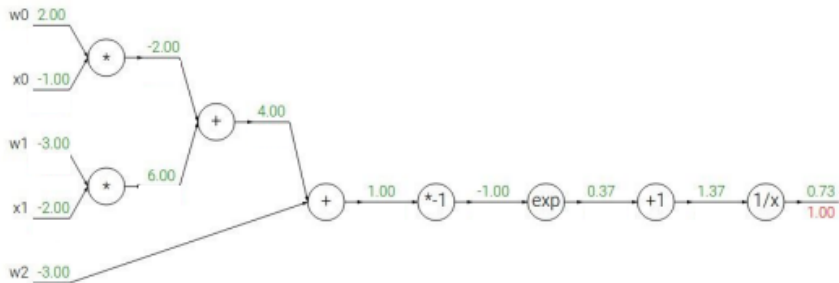
## Back Propagation XIV

### Example2:

Another example: 
$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

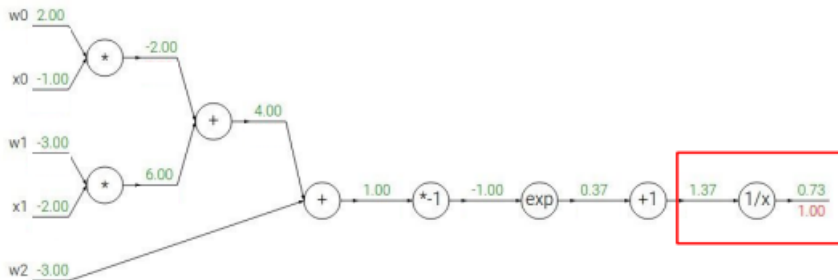


## Back Propagation XV



|               |               |                       |  |                      |               |                          |
|---------------|---------------|-----------------------|--|----------------------|---------------|--------------------------|
| $f(x) = e^x$  | $\rightarrow$ | $\frac{df}{dx} = e^x$ |  | $f(x) = \frac{1}{x}$ | $\rightarrow$ | $\frac{df}{dx} = -1/x^2$ |
| $f_a(x) = ax$ | $\rightarrow$ | $\frac{df}{dx} = a$   |  | $f_c(x) = c + x$     | $\rightarrow$ | $\frac{df}{dx} = 1$      |

## Back Propagation XVI



$$f(x) = e^x$$

$\rightarrow$

$$\frac{df}{dx} = e^x$$

$$f_a(x) = ax$$

$\rightarrow$

$$\frac{df}{dx} = a$$

$$f(x) = \frac{1}{x}$$

$\rightarrow$

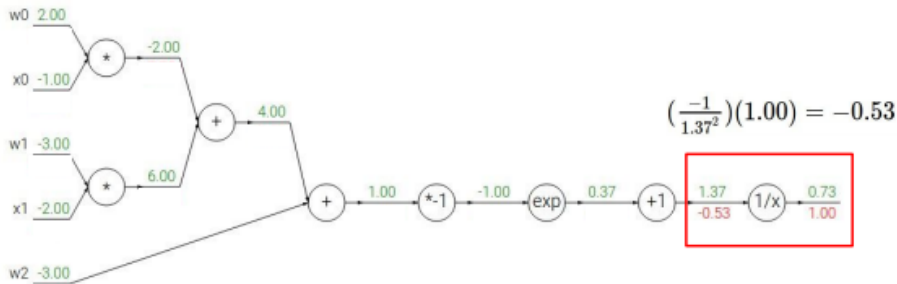
$$\frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x$$

$\rightarrow$

$$\frac{df}{dx} = 1$$

## Back Propagation XVII



|               |               |                       |  |                      |               |                          |
|---------------|---------------|-----------------------|--|----------------------|---------------|--------------------------|
| $f(x) = e^x$  | $\rightarrow$ | $\frac{df}{dx} = e^x$ |  | $f(x) = \frac{1}{x}$ | $\rightarrow$ | $\frac{df}{dx} = -1/x^2$ |
| $f_a(x) = ax$ | $\rightarrow$ | $\frac{df}{dx} = a$   |  | $f_c(x) = c + x$     | $\rightarrow$ | $\frac{df}{dx} = 1$      |

## Normalization vs Standardization I

- ✓ ML that had multiple features spanning varying degrees of magnitude, range, and units.
- ✓ This is a significant obstacle as a few machine learning algorithms are highly sensitive to these features.
- ✓ For example, one feature is entirely in kilograms while the other is in grams, another one is liters, and so on. How can we use these features when they vary so vastly in terms of what they're presenting?
- ✓ This is where the concept of feature scaling comes. It's a crucial part of the data preprocessing stage.

## Normalization vs Standardization II

- ✓ Feature scaling – it improves (significantly) the performance of some machine learning algorithms and does not work at all for others.

## Normalization vs Standardization III

### Normalization and standardization

✓ These are two of the most commonly used feature scaling techniques in machine learning but a level of ambiguity exists in their understanding. When should you use which technique?

### Why Should we Use Feature Scaling?

✓ The first question we need to address – why do we need to scale the variables in our dataset? Some machine learning algorithms are sensitive to feature scaling while others are virtually invariant to it.

### Gradient Descent Based Algorithms

- ✓ Machine learning algorithms like linear regression, logistic regression, neural network, etc. that use gradient descent as an optimization technique require data to be scaled.
- ✓ The presence of feature value  $X$  in the formula will affect the step size of the gradient descent. The difference in ranges of features will cause different step sizes for each feature.
- ✓ To ensure that the gradient descent moves smoothly towards the minima and that the steps for gradient descent are updated at the same rate for all the features, we scale the data before feeding it to the model.

## Normalization vs Standardization V

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m \left( h_{\theta} \left( x^{(i)} \right) - y^{(i)} \right) x_j^{(i)} \quad (3)$$

Having features on a similar scale can help the gradient descent converge more quickly towards the minima.

### Distance-Based Algorithms

## Normalization vs Standardization VI

Distance algorithms like KNN, K-means, and SVM are most affected by the range of features. This is because behind the scenes they are using distances between data points to determine their similarity.

For example, let's say we have data containing high school CGPA scores of students (ranging from 0 to 5) and their future incomes (in thousands Rupees):

## Normalization vs Standardization VII

|   | Student | CGPA | Salary '000 |
|---|---------|------|-------------|
| 0 | 1       | 3.0  | 60          |
| 1 | 2       | 3.0  | 40          |
| 2 | 3       | 4.0  | 40          |
| 3 | 4       | 4.5  | 50          |
| 4 | 5       | 4.2  | 52          |

Since both the features have different scales, there is a chance that higher weightage is given to features with higher magnitude. This will impact the performance of the machine learning algorithm and obviously, we do not want our algorithm to be biased towards one feature.

## Normalization vs Standardization VIII

Therefore, we scale our data before employing a distance based algorithm so that all the features contribute equally to the result.

|   | Student | CGPA      | Salary '000 |
|---|---------|-----------|-------------|
| 0 | 1       | -1.184341 | 1.520013    |
| 1 | 2       | -1.184341 | -1.100699   |
| 2 | 3       | 0.416120  | -1.100699   |
| 3 | 4       | 1.216350  | 0.209657    |
| 4 | 5       | 0.736212  | 0.471728    |

## Normalization vs Standardization IX

The effect of scaling is conspicuous when we compare the Euclidean distance between data points for students A and B, and between B and C, before and after scaling as shown below:

$$\text{Distance AB before scaling} = \sqrt{(40 - 60)^2 + (3 - 3)^2} = 20$$

$$\text{Distance BC before scaling} = \sqrt{(40 - 40)^2 + (4 - 3)^2} = 1$$

$$\text{Distance AB after scaling} = \sqrt{(1.1 - 1.5)^2 + (1.18 - 1.18)^2} = 2.6$$

$$\text{Distance BC after scaling} = \sqrt{(1.1 - 1.1)^2 + (0.41 - 1.18)^2} = 1.59$$

Scaling has brought both the features into the picture and the distances are now more comparable than they were before we applied scaling.

## Normalization vs Standardization X

### Tree-Based Algorithms

Tree-based algorithms, on the other hand, are fairly insensitive to the scale of the features. Think about it, a decision tree is only splitting a node based on a single feature. The decision tree splits a node on a feature that increases the homogeneity of the node. This split on a feature is not influenced by other features.

So, there is virtually no effect of the remaining features on the split. This is what makes them invariant to the scale of the features!

## Normalization vs Standardization XI

### What is Normalization?

✓ Normalization is a scaling technique in which values are shifted and rescaled so that they end up ranging between 0 and 1. It is also known as Min-Max scaling.

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}} \quad (4)$$

Here,  $X_{max}$  and  $X_{min}$  are the maximum and the minimum values of the feature respectively.

## Normalization vs Standardization XII

- When the value of  $X$  is the minimum value in the column, the numerator will be 0, and hence  $X'$  is 0
- On the other hand, when the value of  $X$  is the maximum value in the column, the numerator is equal to the denominator and thus the value of  $X'$  is 1
- If the value of  $X$  is between the minimum and the maximum value, then the value of  $X'$  is between 0 and 1

## Normalization vs Standardization XIII

### What is Standardization?

- ✓ Standardization is another scaling technique where the values are centered around the mean with a unit standard deviation.
- ✓ This means that the mean of the attribute becomes zero and the resultant distribution has a unit standard deviation.

$$X' = \frac{X - \mu}{\sigma} \quad (5)$$

## Normalization vs Standardization XIV

$\mu$  is the mean of the feature values and  $\sigma$  is the standard deviation of the feature values. **The values are not restricted to a particular range.**

### The Big Question – Normalize or Standardize?

- ✓ Normalization is good to use when you know that the distribution of your data does not follow a Gaussian distribution. This can be useful in algorithms that do not assume any distribution of the data like K-Nearest Neighbors and Neural Networks.
- ✓ Standardization, on the other hand, can be helpful in cases where the data follows a Gaussian distribution. However, this does not have to be necessarily true.

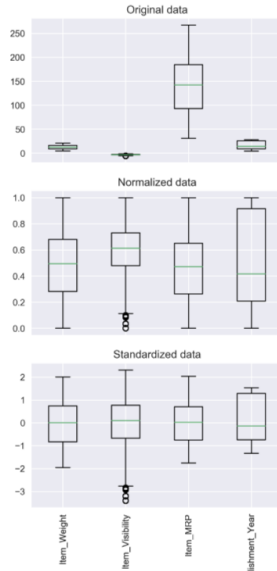
## Normalization vs Standardization XVI

- ✓ Also, unlike normalization, standardization does not have a bounding range. So, even if you have outliers in your data, they will not be affected by standardization.
- ✓ However, the choice of using normalization or standardization will depend on your problem and the machine learning algorithm you are using.
- ✓ There is no hard and fast rule to tell you when to normalize or standardize your data.
- ✓ You can always start by fitting your model to raw, normalized and standardized data and compare the performance for best results.

## Normalization vs Standardization XVII

✓ It is a good practice to fit the scaler on the training data and then use it to transform the testing data. This would avoid any data leakage during the model testing process. Also, the scaling of target values is generally not required.

# Normalization vs Standardization XVIII

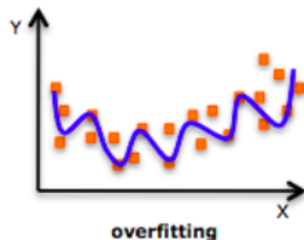
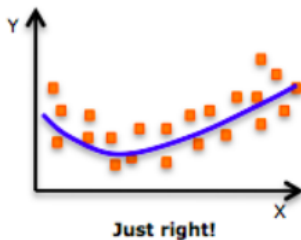
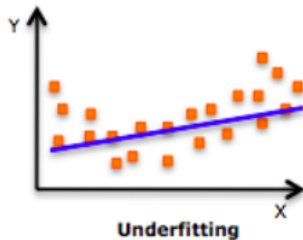


## Regularization Techniques in Deep Learning I

- ✓ One of the most common problems data science professionals face is to avoid overfitting.
- ✓ A situation where a model performed exceptionally well on train data but was not able to predict test data.
- ✓ Avoiding overfitting can single-handedly improve our model's performance.
- ✓ Regularization helps in overcoming the same problem.

## Regularization Techniques in Deep Learning II

### What is Regularization?

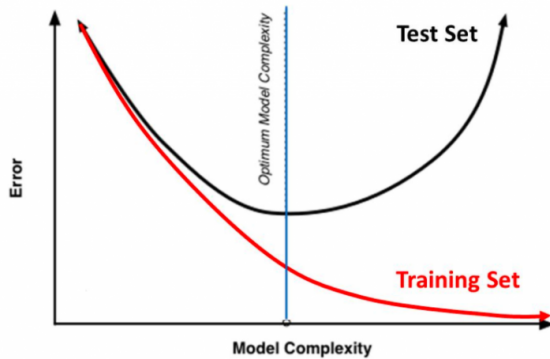


## Regularization Techniques in Deep Learning III

- ✓ As we move towards the right in this image, the model tries to learn too well the details and the noise from the training data, which ultimately results in poor performance on the unseen data.
- ✓ In other words, while going towards the right, the complexity of the model increases such that the training error reduces but the testing error doesn't.

# Regularization Techniques in Deep Learning IV

## Training Vs. Test Set Error

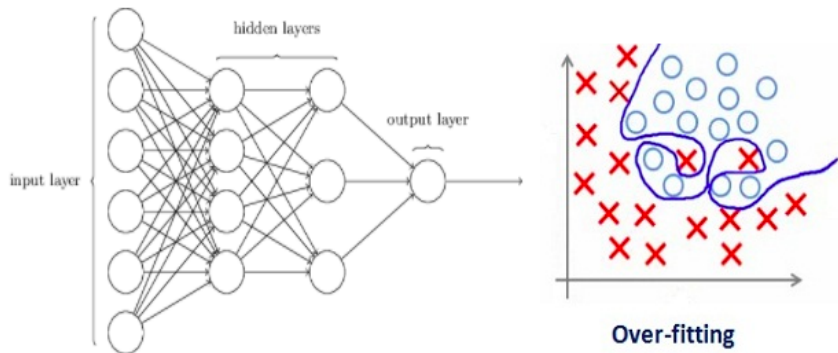


## Regularization Techniques in Deep Learning V

- ✓ A neural network makes model more prone to overfitting.
- ✓ Regularization is a technique which makes slight modifications to the learning algorithm such that the model generalizes better. This in turn improves the model's performance on the unseen data as well.

## Regularization Techniques in Deep Learning VI

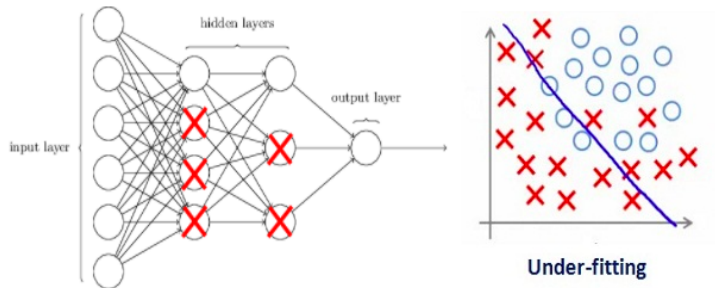
**How does Regularization help reduce Overfitting?**



## Regularization Techniques in Deep Learning VII

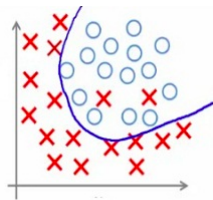
- ✓ In machine learning, a fair idea that regularization penalizes the coefficients. In deep learning, it actually penalizes the weight matrices of the nodes.
- ✓ Assume that our regularization coefficient is so high that some of the weight matrices are nearly equal to zero.

## Regularization Techniques in Deep Learning VIII



## Regularization Techniques in Deep Learning IX

- ✓ This will result in a much simpler linear network and slight underfitting of the training data.
- ✓ Such a large value of the regularization coefficient is not that useful. We need to optimize the value of regularization coefficient in order to obtain a well-fitted model as shown in the image below.



Appropriate-fitting

## Regularization Techniques in Deep Learning X

### Different Regularization Techniques in Deep Learning

✓ A regression model that uses L1 regularization technique is called Lasso Regression and model which uses L2 is called Ridge Regression.

#### L2 & L1 regularization

✓ L1 and L2 are the most common types of regularization. These update the general cost function by adding another term known as the *regularization term*.

*Cost function = Loss (say, binary cross entropy) + Regularization term*

✓ Due to the addition of this regularization term, the values of weight matrices decrease because it assumes that a neural network with smaller

## Regularization Techniques in Deep Learning XI

weight matrices leads to simpler models. Therefore, it will also reduce overfitting to quite an extent.

However, this regularization term differs in L2.

## Regularization Techniques in Deep Learning XII

In L2:

$$\text{Cost function} = \text{Loss} + \frac{\lambda}{2m} * \sum \|w\|^2 \quad (6)$$

Here,  $\lambda$  is the regularization parameter. It is the hyperparameter whose value is optimized for better results.

✓ L2 regularization is also known as weight decay as it forces the weights to decay towards zero (but not exactly zero).

## Regularization Techniques in Deep Learning XIII

In L1:

$$\text{Cost function} = \text{Loss} + \frac{\lambda}{2m} * \sum \|w\| \quad (7)$$

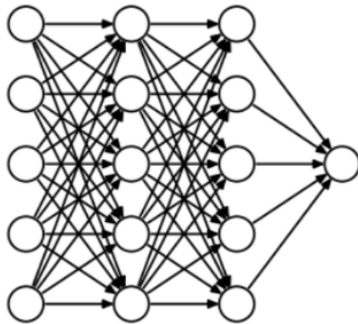
✓ In this, we penalize the absolute value of the weights. Unlike L2, the weights may be reduced to zero here. Hence, it is very useful when we are trying to compress our model. Otherwise, we usually prefer L2 over it.

## Regularization Techniques in Deep Learning XIV

### Dropout:

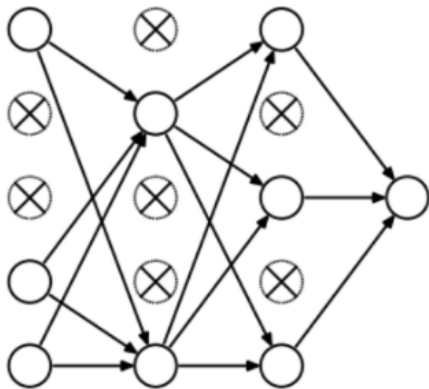
- ✓ This is the one of the most interesting types of regularization techniques. It also produces very good results and is consequently the most frequently used regularization technique in the field of deep learning.
- ✓ To understand dropout, let's say our neural network structure is akin to the one shown below:

## Regularization Techniques in Deep Learning XV



✓ At every iteration, it randomly selects some nodes and removes them along with all of their incoming and outgoing connections as shown below. 🏠

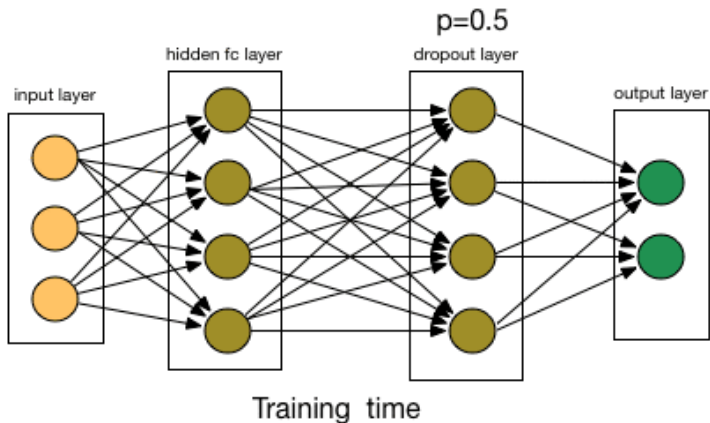
## Regularization Techniques in Deep Learning XVI



## Regularization Techniques in Deep Learning XVII

- ✓ So each iteration has a different set of nodes and this results in a different set of outputs. It can also be thought of as an ensemble technique in machine learning.
- ✓ Ensemble models usually perform better than a single model as they capture more randomness. Similarly, dropout also performs better than a normal neural network model.
- ✓ This probability of choosing how many nodes should be dropped is the hyperparameter of the dropout function. Dropout can be applied to both the hidden layers as well as the input layers.

## Regularization Techniques in Deep Learning XVIII



## Regularization Techniques in Deep Learning XIX

✓ *Due to these reasons, dropout is usually preferred when we have a large neural network structure in order to introduce more randomness.*

**Data Augmentation:** The simplest way to reduce overfitting is to increase the size of the training data. In machine learning, it is not possible to increase the size of training data as the labeled data was too costly.

✓ For example, dealing with images. In this case, there are a few ways of increasing the size of the training data – rotating the image, flipping, scaling, shifting, etc.

✓ In the below image, some transformation has been done on the handwritten digits dataset.

## Regularization Techniques in Deep Learning XX



✓ This technique is known as data augmentation. This usually provides a big leap in improving the accuracy of the model. It can be considered as a mandatory trick in order to improve the predictions.

**Early stopping:** Early stopping is a kind of cross-validation strategy where we keep one part of the training set as the validation set. When we see that the performance on the validation set is getting worse, we immediately stop the training on the model. This is known as early stopping.

## Regularization Techniques in Deep Learning XXI



✓ In the above image, we will stop training at the dotted line since after that our model will start overfitting on the training data.

## Weight Initialization I

### Why do we need it?

✓ There are three main reasons why it has been used.

- ➊ One of the reasons is that it can avoid Vanishing, exploding gradients problem in the late part of training.
- ➋ Vanishing gradients problems refer to the problem which occurs when weights don't change during the training (occurs when weights are initialized very much less than 1).
- ➌ Exploding gradients problem refers to the problem which occurs when weights keep on increasing rapidly (occurs when weights are initialized greater than 1)

## Weight Initialization II

- ④ Another reason is it can speed up the training process as it can learn faster with optimal initialization.

### Key Points:

- ① Weights should be small
- ② Weights should not be same
- ③ Weights should follow the same distribution
- ④ Weights should have good variance

## Weight Initialization III

### Different ways to initialize the weight

#### ① Uniform Distribution:

$$W_{ij} = \text{Uniform}\left[\frac{-1}{\sqrt{f_n}}, \frac{1}{\sqrt{f_n}}\right], f_n \text{ means number of inputs.}$$

**This works well for the sigmoid activation function.**

#### ② Xavier/Gorat Distribution:

##### ① Here the weight is initialization using the normal distribution:

$W_{ij} = N(\text{mean}, \text{std})$ ,  $\text{mean} = 0$ ,  $\text{std} = \sqrt{\frac{2}{(f_i + f_o)}}$ ,  $f_i$  and  $f_o$  are number of inputs and outputs, respectively.

## Weight Initialization IV

- ② Here the weight is initialization using a uniform distribution:

$W_{ij} = \text{Uniform} \left[ -\frac{\sqrt{6}}{\sqrt{(f_i + f_o)}}, \frac{\sqrt{6}}{\sqrt{(f_i + f_o)}} \right]$ ,  $f_i$  and  $f_o$  are number of inputs and outputs, respectively.

- ③ He init:

- ① He uniform : Here the weight is initialization using the normal distribution.

$W_{ij} = \text{Uniform} \left[ -\sqrt{\frac{6}{f_i}}, \sqrt{\frac{6}{f_i}} \right]$ ,  $f_i$  is number of inputs.

- ② He Normal : Here the weight is initialization using the normal distribution.

$W_{ij} \sim N(\text{mean}, \text{std})$ ,  $\text{mean} = 0$ ,  $\text{std} = \sqrt{\frac{2}{f_i}}$ ,  $f_i$  is number of inputs.

**This works well for the ReLU activation function.**